

# SI1001 Teoría de la Computación

## Lógica de Hoare

Andrés Sicard Ramírez

Universidad EAFIT

Semestre 2025-2

# Preliminares

- El texto guía para estas diapositivas es (Grassman y Tremblay 1996, cap. 9).
- Los números asignados a las definiciones, ejemplos, ejercicios, figuras, páginas, secciones, tablas y teoremas en estas diapositivas corresponden a los números asignados en el texto guía.

# Conceptos preliminares

## Descripción

Emplearemos la lógica de Hoare para verificar programas de un subconjunto del lenguaje imperativo PASCAL:

- (i) operadores aritméticos,
- (ii) funciones de la biblioteca estándar,
- (iii) bloques **begin-end** ,
- (iv) instrucciones de asignación ( **:=** ),
- (v) instrucciones de secuenciación ( **;** ),
- (vi) instrucciones **if-then** y **if-then-else** ,
- (vii) instrucciones **while** .

# Conceptos preliminares

## Descripción

El **estado de un programa** consiste del valor de la variables utilizadas por el programa.

# Conceptos preliminares

## Descripción

El **estado de un programa** consiste del valor de la variables utilizadas por el programa.

## Descripción

Las **sentencias (proposiciones)** de la lógica de Hoare son sentencias (proposiciones) de la lógica de predicados de primer orden en las variables del programa.

# Conceptos preliminares

## Notación

Para escribir las sentencias emplearemos las siguientes conectivas y constantes lógicas:

**V** (verdadero)

**F** (falso)

$\neg$  (negación)

$\wedge$  (conjunción)

$\vee$  (disyunción)

$\Rightarrow$  (condicional, implicación material)

$\Leftrightarrow$  (bicondicional, equivalencia material)

$=$  (igualdad)

# Conceptos preliminares

## Notación

$A \equiv> B$  (implicación lógica, es decir,  $A \Rightarrow B$  es una tautología)

$A \equiv B$  (equivalencia lógica, es decir,  $A \Leftrightarrow B$  es una tautología)

# Conceptos preliminares

Tabla 1.16 (Equivalencias lógicas)

|                                         |                       |
|-----------------------------------------|-----------------------|
| $P \vee \neg P \equiv \mathbf{V}$       | Ley de medio excluido |
| $P \wedge \neg P \equiv \mathbf{F}$     | Ley de contradicción  |
| $P \wedge \mathbf{V} \equiv P$          | Leyes de identidad    |
| $P \vee \mathbf{F} \equiv P$            |                       |
| $P \vee \mathbf{V} \equiv \mathbf{V}$   | Leyes de dominación   |
| $P \wedge \mathbf{F} \equiv \mathbf{F}$ |                       |
| $P \vee P \equiv P$                     | Leyes de idempotencia |
| $P \wedge P \equiv P$                   |                       |
| $\neg(\neg P) \equiv P$                 | Ley de doble negación |

(continua en la próxima diapositiva)



# Conceptos preliminares

Tabla 1.16 (Equivalencias lógicas (continuación))

|                                                                                                                          |                     |
|--------------------------------------------------------------------------------------------------------------------------|---------------------|
| $P \vee Q \equiv Q \vee P$<br>$P \wedge Q \equiv Q \wedge P$                                                             | Leyes conmutativas  |
| $(P \vee Q) \vee R \equiv P \vee (Q \vee R)$<br>$(P \wedge Q) \wedge R \equiv P \wedge (Q \wedge R)$                     | Leyes asociativas   |
| $P \vee (Q \wedge R) \equiv (P \vee Q) \wedge (P \vee R)$<br>$P \wedge (Q \vee R) \equiv (P \wedge Q) \vee (P \wedge R)$ | Leyes distributivas |
| $\neg(P \wedge Q) \equiv \neg P \vee \neg Q$<br>$\neg(P \vee Q) \equiv \neg P \wedge \neg Q$                             | Leyes de De Morgan  |

# Conceptos preliminares

## Notación

Un argumento válido con premisas  $A_1, A_2, \dots, A_n$  y conclusión  $B$ , se denotará por:

$$A_1, A_2, \dots, A_n \models B.$$

# Conceptos preliminares

Tabla 1.25 (Leyes de inferencia)

|                                                                |                                           |
|----------------------------------------------------------------|-------------------------------------------|
| $A, B \models A \wedge B$                                      | Ley de combinación                        |
| $A \wedge B \models B$                                         | Ley de simplificación                     |
| $A \wedge B \models A$                                         | Ley de simplificación (variante)          |
| $A \models A \vee B$                                           | Ley de adición                            |
| $B \models A \vee B$                                           | La de adición (variante)                  |
| $A, A \Rightarrow B \models B$                                 | Modus ponens                              |
| $\neg B, A \Rightarrow B \models \neg A$                       | Modus tollens                             |
| $A \Rightarrow B, B \Rightarrow C \models A \Rightarrow C$     | Silogismo hipotético                      |
| $A \vee B, \neg A \Rightarrow \models B$                       | Silogismo disyuntivo                      |
| $A \vee B, \neg B \Rightarrow \models A$                       | Silogismo disyuntivo (variante)           |
| $A \Rightarrow B, \neg A \Rightarrow B \models B$              | Ley de casos                              |
| $A \Leftrightarrow B \models A \Rightarrow B$                  | Eliminación de la equivalencia            |
| $A \Leftrightarrow B \models B \Rightarrow A$                  | Eliminación de la equivalencia (variante) |
| $A \Rightarrow B, B \Rightarrow A \models B \Leftrightarrow A$ | Introducción de la equivalencia           |
| $A, \neg A \models B$                                          | Ley de inconsistencia                     |

# Conceptos preliminares

## Definición 9.1

Una **aserción** (**afirmación**) es una sentencia referente a un estado de programa.

# Conceptos preliminares

## Definición 9.1

Una **aserción** (**afirmación**) es una sentencia referente a un estado de programa.

## Notación

Las aserciones se escribirán entre llaves. Es decir,  $\{A\}$  denota que  $A$  es una aserción.

# Conceptos preliminares

## Definición 9.2

«Si  $C$  es una parte de un código, entonces cualquier aserción  $\{P\}$  se denomina **precondición** de  $C$  si  $\{P\}$  sólo implica el estado inicial. Cualquier aserción  $\{Q\}$  se denomina **postcondición** [de  $C$ ] si  $\{Q\}$  sólo implica el estado final. Si  $C$  tiene como precondición a  $\{P\}$  y como postcondición a  $\{Q\}$ , se escribe  $\{P\} C \{Q\}$ . La terna  $\{P\} C \{Q\}$  se denomina **terna de Hoare**.»

# Conceptos preliminares

## Ejemplo (terna de Hoare)

Para la terna de Hoare

$$\{y \neq 0\} \ x := 1/y \ \{x = 1/y\},$$

- la precondition  $y \neq 0$  afirma que el valor de  $y$  es diferente de 0,
- la postcondition  $x = 1/y$  afirma que el valor de  $x$  es  $1/y$ ,
- y el programa con la precondition y postcondition anteriores es la instrucción  $x := 1/y$ .

# Conceptos preliminares

## Ejemplo (precondición vacía)

La terna de Hoare

$$\{\} \ a \ := \ b \ \{a = b\},$$

tiene una precondición vacía  $\{\}$  la cual se interpreta como «verdadera para todos los estados del programa».



# Conceptos preliminares

## Pregunta

Las instrucciones `h := a; a := b; b := h` intercambian los valores de las variables `a` y `b`. ¿Cómo escribir una postcondición para estas instrucciones?

# Conceptos preliminares

Métodos para establecer la distinción entre los valores de las variables en el estado inicial y en el estado final

(i) Uso de subíndices

El subíndice  $\alpha$  se empleará para los valores iniciales de las variables y el subíndice  $\omega$  se empleará para los valores finales de las variables.

(ii) Uso de variables ocultas

Se utilizan variables que no aparecen en el código para almacenar los valores iniciales de las variables.

# Conceptos preliminares

## Ejemplos (diferenciando el estado inicial y el estado final)

Ternas de Hoare para las instrucciones que intercambian los valores de dos variables.

1. Empleando los subíndices  $\alpha$  y  $\omega$

$$\{\} \text{ h } := \text{ a; a } := \text{ b; b } := \text{ h } \{ (a_\omega = b_\alpha) \wedge (b_\omega = a_\alpha) \}.$$

2. Empleando las variables ocultas  $A$  y  $B$

$$\{a = A, b = B\} \text{ h } := \text{ a; a } := \text{ b; b } := \text{ h } \{a = B, b = A\}.$$

# Conceptos preliminares

## Ejemplo

Una terna de Hoare para una instrucción de asignación.

$$\{\} \text{ a } := \text{ b } \{a = b, b = b_{\alpha}\},$$

# Conceptos preliminares

## Ejemplo

Una terna de Hoare para una instrucción **if-then-else**.

$\{$

**if**  $x > y$  **then**  $m := x$  **else**  $m := y$

$\{(x > y \Rightarrow m = x_\alpha) \wedge (\neg(x > y) \Rightarrow m = y_\alpha)\}$ .

# Conceptos preliminares

## Definición 9.3

«Si  $C$  es un código con la precondition  $\{P\}$  y la postcondición  $\{Q\}$ , entonces  $\{P\} C \{Q\}$  se dice que es **parcialmente correcta** si el estado final de  $C$  satisface  $\{Q\}$ , siempre que el estado inicial satisfaga  $\{P\}$ . [El programa]  $C$  también sería parcialmente correcto si no hay estado final debido a que el programa no termina. Si  $\{P\} C \{Q\}$  es parcialmente correcta y  $C$  termina, entonces se dice que  $\{P\} C \{Q\}$  es **totalmente correcta**.»

# Conceptos preliminares

## Observación

En el subconjunto de PASCAL que estamos considerando el problema de la terminación solo está asociado a programas que tengan instrucciones **while** .

# Conceptos preliminares

## Observación

En el subconjunto de PASCAL que estamos considerando el problema de la terminación solo está asociado a programas que tengan instrucciones **while** .

## Observación

Observe que la terna de Hoare  $\{P\} C \{Q\}$  es **parcialmente correcta** «si cada estado inicial posible que satisfaga  $P$  da como resultado un estado final que satisfaga  $Q$ .»



# Conceptos preliminares

## Descripción

La lógica de Hoare (también llamada lógica de Floyd-Hoare) es un sistema formal para construir demostraciones de la corrección parcial de ternas de Hoare  $\{P\} C \{Q\}$  compuesta por axiomas y reglas de inferencia.

# Reglas generales relativas a las precondiciones y postcondiciones

## Definición 9.4

«Si  $R$  y  $S$  son dos aserciones, entonces se dice que  $R$  es **más fuerte** que  $S$  si  $R \Rightarrow S$ . Si  $R$  es más fuerte que  $S$  entonces se dice que  $S$  es **más débil** que  $R$ .»

# Reglas generales relativas a las precondiciones y postcondiciones

## Definición 9.4

«Si  $R$  y  $S$  son dos aserciones, entonces se dice que  $R$  es **más fuerte** que  $S$  si  $R \Rightarrow S$ . Si  $R$  es más fuerte que  $S$  entonces se dice que  $S$  es **más débil** que  $R$ .»

## Ejemplos

Sean  $R$  y  $S$  son dos aserciones.

- $R \wedge S$  es más fuerte que  $R$ .
- $R \vee S$  es más débil que  $R$ .
- $i < 0$  es más fuerte que  $i < 1$ .
- $i > 1$  es más débil que  $i > 0$ .

# Reglas generales relativas a las precondiciones y postcondiciones

Regla de inferencia: Reforzamiento de precondiciones

$$\frac{P' \Rightarrow P \quad \{P\} C \{Q\}}{\{P'\} C \{Q\}}.$$

# Reglas generales relativas a las precondiciones y postcondiciones

## Ejemplo 9.2

Dando por hecho que la siguiente terna de Hoare se cumple

$$\{x \geq 0\} \text{ y } := \text{sqrt}(x) \{y^2 = x\}.$$

Demostrar que  $\{x \geq 1\} \text{ y } := \text{sqrt}(x) \{y^2 = x\}.$

# Reglas generales relativas a las precondiciones y postcondiciones

## Ejemplo 9.2

Dando por hecho que la siguiente terna de Hoare se cumple

$$\{x \geq 0\} \text{ y } := \text{sqrt}(x) \{y^2 = x\}.$$

Demostrar que  $\{x \geq 1\} \text{ y } := \text{sqrt}(x) \{y^2 = x\}$ .

## Demostración

1.  $x \geq 1 \Rightarrow x \geq 0$  (aritmética)
2.  $\{x \geq 0\} \text{ y } := \text{sqrt}(x) \{y^2 = x\}$  (hipótesis)
3.  $\{x \geq 1\} \text{ y } := \text{sqrt}(x) \{y^2 = x\}$  (reforzamiento de precondición en 1 y 2)



# Reglas generales relativas a las precondiciones y postcondiciones

## Pregunta

¿Es preferible construir una terna de Hoare con la precondición más débil posible o con la precondición más fuerte posible?

# Reglas generales relativas a las precondiciones y postcondiciones

## Pregunta

¿Es preferible construir una terna de Hoare con la precondición más débil posible o con la precondición más fuerte posible?

## Respuesta

*«Por regla general, siempre es ventajoso intentar formular la precondición más débil posible que asegure que se cumple una cierta postcondición dada. Cualquier precondición [más] fuerte será automáticamente satisfecha debido al principio de reforzamiento de precondiciones... El programa que tenga la precondición más débil es el programa más general.» (pág. 465)*



# Reglas generales relativas a las precondiciones y postcondiciones

Regla de inferencia: Debilitamiento de postcondiciones

$$\frac{\{P\} C \{Q\} \quad Q \Rightarrow Q'}{\{P\} C \{Q'\}}.$$

# Reglas generales relativas a las precondiciones y postcondiciones

## Ejemplo 9.4

Dada la siguiente terna de Hoare

$$\{a > 4\} \text{ b := a + 1 } \{b > 5, a > 4\}$$

Demostrar que  $\{a > 4\} \text{ b := a + 1 } \{b > 5\}$ .

# Reglas generales relativas a las precondiciones y postcondiciones

## Ejemplo 9.4

Dada la siguiente terna de Hoare

$$\{a > 4\} \text{ b := a + 1 } \{b > 5, a > 4\}$$

Demostrar que  $\{a > 4\} \text{ b := a + 1 } \{b > 5\}$ .

## Demostración

1.  $\{a > 4\} \text{ b := a + 1 } \{b > 5, a > 4\}$  (hipótesis)
2.  $b > 5, a > 4 \Rightarrow (b > 5) \wedge (a > 4)$  (lógica, ley de combinación)
3.  $(b > 5) \wedge (a > 4) \Rightarrow b > 5$  (lógica, ley de simplificación)
4.  $\{a > 4\} \text{ b := a + 1 } \{b > 5\}$  (debilitamiento de postcondición en 1 y 3)



# Reglas generales relativas a las precondiciones y postcondiciones

## Pregunta

¿Es preferible construir una terna de Hoare con la postcondición más débil posible o con la postcondición más fuerte posible?

# Reglas generales relativas a las precondiciones y postcondiciones

## Pregunta

¿Es preferible construir una terna de Hoare con la postcondición más débil posible o con la postcondición más fuerte posible?

## Respuesta

*«Si es posible, se debe intentar encontrar postcondiciones tan específicas o tan fuertes como sea posible, especialmente cuando una parte de un código se utilice en varios lugares diferentes. Si en una aplicación en particular una postcondición más débil resulta aceptable, siempre se puede utilizar el debilitamiento de la postcondición.» (pág. 467)*

# Reglas generales relativas a las precondiciones y postcondiciones

Regla de inferencia: Regla de conjunción

$$\frac{\begin{array}{c} \{P_1\} C \{Q_1\} \\ \{P_2\} C \{Q_2\} \end{array}}{\{P_1 \wedge P_2\} C \{Q_1 \wedge Q_2\} .}$$

# Reglas generales relativas a las precondiciones y postcondiciones

## Ejemplo 9.5

Aplicar las ternas de Hoare

$$\begin{aligned} & \{\} \text{ i := i + 1 } \{i_{\omega} = i_{\alpha} + 1\}, \\ & \{i_{\alpha} > 0\} \text{ i := i + 1 } \{i_{\alpha} > 0\}, \end{aligned}$$

para demostrar que  $\{i_{\alpha} > 0\} \text{ i := i + 1 } \{i_{\omega} > 1\}$ .

(continua en la próxima diapositiva)

# Reglas generales relativas a las precondiciones y postcondiciones

## Demostración

- |     |                                                                                     |                                        |
|-----|-------------------------------------------------------------------------------------|----------------------------------------|
| 1.  | $\{\} \ i := i + 1 \ \{i_w = i_\alpha + 1\}$                                        | (hipótesis)                            |
| 2.  | $\{i_\alpha > 0\} \ i := i + 1 \ \{i_\alpha > 0\}$                                  | (hipótesis)                            |
| 3.  | $\{i_\alpha > 0\} \ i := i + 1$<br>$\{(i_w = i_\alpha + 1) \wedge (i_\alpha > 0)\}$ | (regla de conjunción en 1 y 2)         |
| 4.  | $(i_w = i_\alpha + 1) \wedge (i_\alpha > 0)$                                        | (supuesto)                             |
| 5.  | $(i_\alpha = i_w - 1) \wedge (i_\alpha > 0)$                                        | (aritmética)                           |
| 6.  | $(i_\alpha = i_w - 1) \wedge (i_w - 1 > 0)$                                         | (sustitución)                          |
| 7.  | $i_w - 1 > 0$                                                                       | (lógica, ley de simplificación en 6)   |
| 8.  | $i_w > 1$                                                                           | (aritmética)                           |
| 9.  | $(i_w = i_\alpha + 1) \wedge (i_\alpha > 0) \Rightarrow (i_w > 1)$                  | (demostración condicional en 4–8)      |
| 10. | $\{i_\alpha > 0\} \ i := i + 1 \ \{i_w > 1\}$                                       | (debilitamiento de postcond. en 3 y 9) |



# Reglas generales relativas a las precondiciones y postcondiciones

Regla de inferencia: Regla de disyunción

$$\frac{\begin{array}{c} \{P_1\} C \{Q_1\} \\ \{P_2\} C \{Q_2\} \end{array}}{\{P_1 \vee P_2\} C \{Q_1 \vee Q_2\}}.$$

## Referencias



Winfried Karl Grassman y Jean-Paul Tremblay (1996). Matemáticas Discretas y Lógica. Una Perspectiva desde la Ciencia de la Computación. Trad. por Rafael García-Bermejo, María Luisa Díez Platas y Vivian de los Ángeles Fernández Vásquez. Prentice Hall, 1996 (vid. [pág. 2](#)).